

Control de Gastos Personales

Propuesta de app funcional en React Native + Firebase

Seccion	Descripcion
Objetivo	Registrar ingresos y gastos, consultar reportes, adjuntar comprobantes y exportar datos en CSV por mail.
Tecnologia	React Native con Expo, Firebase Authentication, Cloud Firestore y Firebase Storage.
Servicio del S.O.	Camara del dispositivo para cargar foto del comprobante. Opcional: compartir CSV por mail mediante Intent/Share.

1. Resumen general

- La app permite que cada usuario gestione sus movimientos financieros personales desde el celular.
- Cumple con CRUD completo: crear, listar, editar y eliminar movimientos.
- Muestra estadísticas como ingresos, gastos, balance y gastos por categoría.
- Usa Firebase como base de datos y almacenamiento remoto.
- Permite exportar reportes en CSV y enviarlos por mail o compartirlos.
- Usa la cámara para adjuntar una foto del ticket o comprobante.

Alcance para el trabajo practico

- MVP funcional con login, dashboard, listado, formulario, detalle, estadísticas y exportación.
- Persistencia real en Firebase, no datos simulados.
- Validaciones básicas y manejo correcto de errores.
- Diseño simple, claro y user friendly.

2. Stack tecnico propuesto

Componente	Eleccion	Uso
Frontend	React Native + Expo	Desarrollo mobile para Android con componentes reutilizables.
Navegacion	React Navigation	Manejo de pantallas: Stack y Tabs.
Autenticacion	Firebase Auth	Registro e inicio de sesion con email/password.
Base de datos	Cloud Firestore	Guardar usuarios, movimientos y categorias.
Archivos	Firebase Storage	Guardar imagenes de comprobantes.
Camara	expo-image-picker o expo-camera	Tomar foto o elegir imagen de la galeria.
Exportacion	expo-file-system + expo-sharing	Crear CSV y compartirlo por mail/apps.

3. Pantallas concretas

Pantalla	Funcion	Elementos principales
LoginScreen	Permite iniciar sesion.	Email, password, boton ingresar, link a registro.
RegisterScreen	Permite crear una cuenta.	Nombre, email, password, confirmacion, boton registrarse.
DashboardScreen	Resumen financiero del usuario.	Balance, ingresos, gastos, accesos rapidos, ultimos movimientos.
TransactionsScreen	Listado de movimientos.	Buscador, filtros por tipo/categoria/fecha, FlatList, boton agregar.
TransactionFormScreen	Alta y edicion.	Titulo, monto, tipo, categoria, fecha, nota, foto, guardar.
TransactionDetailScreen	Detalle de un movimiento.	Datos completos, imagen, editar, eliminar.
StatsScreen	Reportes y estadisticas.	Totales, balance, gastos por categoria, movimientos del mes.
ExportScreen	Exportacion.	Selector de rango, generar CSV, compartir por mail.

Pantalla	Funcion	Elementos principales
ProfileScreen	Datos del usuario.	Nombre, email, cerrar sesion.

4. Navegacion sugerida

- AuthStack: LoginScreen y RegisterScreen.
- MainTabs: DashboardScreen, TransactionsScreen, StatsScreen, ExportScreen y ProfileScreen.
- TransactionStack: TransactionsScreen, TransactionFormScreen y TransactionDetailScreen.
- El boton principal para agregar movimiento debe estar visible desde el listado o dashboard.

5. Modelo de datos en Firebase

Colecciones principales

Coleccion	Documento	Campos
users	{userId}	name, email, createdAt
transactions	{transactionId}	userId, title, amount, type, categoryId, categoryName, date, note, imageUrl, createdAt, updatedAt
categories	{categoryId}	userId, name, type, color, icon

Ejemplo de transaccion

Campo	Tipo	Ejemplo
title	string	Supermercado
amount	number	12500
type	string	expense o income
categoryName	string	Comida
date	timestamp	2026-04-16
note	string	Compra semanal
imageUrl	string	URL del comprobante en Firebase Storage
userId	string	ID del usuario autenticado

6. CRUD requerido

Operacion	Implementacion
Create	Crear una transaccion desde el formulario y guardarla en Firestore. Si tiene imagen, subirla primero a Firebase Storage.
Read	Listar movimientos filtrados por userId y ordenados por fecha descendente.
Update	Editar datos de una transaccion existente y actualizar updatedAt.
Delete	Eliminar el documento de Firestore. Opcional: eliminar tambien la imagen del Storage.

7. Reportes y estadisticas

- Total de ingresos del mes.

- Total de gastos del mes.
- Balance: ingresos menos gastos.
- Cantidad de movimientos registrados.
- Gastos agrupados por categoria.
- Mayor gasto registrado.
- Promedio de gasto mensual o diario.

Para simplificar, las estadísticas pueden calcularse en la app a partir de las transacciones descargadas de Firestore.

8. Exportacion CSV y mail

- La pantalla ExportScreen permite elegir un rango de fechas.
- La app genera un archivo CSV con columnas: fecha, tipo, categoria, titulo, monto y nota.
- El archivo se guarda temporalmente usando expo-file-system.
- Luego se comparte usando expo-sharing para enviarlo por mail, WhatsApp, Drive u otra app disponible.
- Esto cumple con el requisito de exportacion CSV/mail.

9. Uso de servicios del sistema operativo

Servicio	Uso en la app	Permiso
Camara	Tomar foto del comprobante de un gasto.	CAMERA
Galeria	Elegir imagen ya existente.	READ_MEDIA_IMAGES o equivalente segun version.
Compartir archivo	Enviar CSV por mail u otra app.	No siempre requiere permiso explicito con Expo Sharing.
Internet	Conectar con Firebase.	INTERNET

10. Validaciones y excepciones

- El titulo no puede estar vacio.
- El monto debe ser numerico y mayor a cero.
- La categoria debe estar seleccionada.
- Debe validarse si el usuario concedio permiso de camara.
- Los errores de Firebase deben mostrarse con mensajes claros.
- No se debe dejar la app en blanco si falla la conexion.

11. Estructura de carpetas

Carpeta	Contenido
src/screens	Pantallas principales de la app.
src/components	Componentes reutilizables: cards, inputs, botones.
src/navigation	Configuracion de React Navigation.
src/services	Servicios de Firebase, exportacion y camara.
src/models	Tipos/modelos de datos.

Carpeta	Contenido
src/constants	Constantes de rutas, colores, colecciones y textos.
src/utills	Funciones auxiliares: formatear moneda, fechas, CSV.

12. Buenas practicas solicitadas

- Usar nombres claros y entendibles en componentes, funciones y archivos.
- No hardcodear textos repetidos: usar constantes cuando corresponda.
- Separar logica de Firebase en servicios, no mezclar todo dentro de las pantallas.
- Manejar errores con try/catch en operaciones asincronicas.
- Pedir permisos antes de usar camara o galeria.
- Mantener componentes chicos y reutilizables.
- Diseño consistente: mismos colores, espaciados, botones y tipografias.

13. Resumen para presentar al grupo

La aplicacion propuesta es un Control de Gastos Personales desarrollado en React Native. Permitira que cada usuario registre ingresos y gastos, los edite, elimine y consulte desde una base de datos en Firebase. La app incluirea reportes y estadisticas, exportacion de movimientos a CSV y uso de la camara para adjuntar comprobantes. El objetivo es entregar una aplicacion funcional, clara, user friendly y alineada a los requerimientos minimos del trabajo practico.